

You’ve got a really clear picture of what you want—this is less “dumb little editor” and more “focused, single-user tool with real constraints.” Let’s line up your options against those constraints.

High-level comparison

| Option              | JSON handling         | DDS display        | Packaging as single folder exe | Dev speed (for you) |
|---------------------|-----------------------|--------------------|--------------------------------|---------------------|
| Node.js + NW.js     | Excellent (native)    | Needs DDS decoder  | Yes (nw.exe + app folder)      | Fast                |
| C++ + Irrlicht      | Manual / library JSON | Native DDS support | Yes (exe + dlls)               | Medium              |
| C++ + SDL2 + OpenGL | Manual / library JSON | Needs DDS decoder  | Yes                            | Medium/High         |

Direct recommendation

Given everything you wrote—your comfort with C++, your desire to keep it small, offline, and self-contained, and the need to show DDS mech images next to text—I’d lean toward:

C++ + Irrlicht + a JSON library (e.g., nlohmann/json)

Reasons:

- **DDS is “just works”** in Irrlicht; no need to fight Chromium/WebGL or custom decoders.
- **You already know Irrlicht and OpenGL concepts**, so UI + 2D sprites + text is straightforward.
- **No browser stack, no NW.js runtime**, just an `.exe` and a few `.dll`s in your mod folder.
- JSON parsing in C++ is easy with a good library; the hard part is your data model, not the language.

If you really want to stay in JavaScript, NW.js is still viable—but you’ll pay a complexity tax for DDS.

Option 1: C++ + Irrlicht editor

Core pieces

- **Rendering/UI:**
  - Use Irrlicht in a simple 2D GUI style:
    - One main window.
    - Left panel: list of vanilla mechs (parsed from game JSON).
    - Middle/right panel: current custom mech (slots, weapons, equipment).
    - Side/preview area: DDS portrait/thumbnail of the mech.
  - Use Irrlicht’s GUI elements (`IGUIEnvironment`, list boxes, edit boxes, buttons) for a basic editor UI.

- **JSON handling:**

- Pull in **nlohmann/json** (single header) or similar.
- Build small C++ structs for:
  - `WeaponDef`, `AmmoBoxDef`, `PilotDef`, `MechDef`, `ChassisDef`, `MoveDef`, etc.
- Write:
  - **Load functions:** parse vanilla JSON + your mod JSON into those structs.
  - **Save functions:** serialize your structs back into ModTek-compatible JSON.

- **Data flow:**

1. On startup, scan:
  - Vanilla game data folders (for base mech/chassis/move defs).
  - Your mod folder (for custom weapons/equipment/etc.).
2. Build in-memory catalogs:
  - "Available weapons", "available equipment", "available chassis", etc.
3. UI lets you:
  - Pick a base vanilla mech.
  - Clone it into a "custom mech" definition.
  - Assign weapons/equipment from your custom catalog.
4. Save to:
  - `mod_folder/CustomMechs/mechdef_*.json`
  - `mod_folder/CustomMechs/chassisdef_*.json`
  - `mod_folder/CustomMechs/movedef_*.json` (if needed).

- **Packaging:**

- Put in your mod folder:
  - `Editor/`
    - `bt_mech_editor.exe`
    - `Irrlicht.dll` (from MSYS2)
    - `json.hpp` (in `src/` only)
    - `config.json` (paths to game + mod)
  - `src/` (for GitHub, optional for end-user)
- ModTek ignores `Editor/`, but it's convenient for you and anyone using your mod.

## Option 2: Node.js + NW.js editor

If you decide you'd rather stay in JavaScript:

- **Pros:**
  - JSON is native and trivial.
  - UI can be HTML/CSS, which is flexible.
  - **Node.js** gives you filesystem access to game + mod folders.
- **Cons:**
  - Chromium **cannot load DDS directly**—you must:
    - Use a DDS decoder (JS or WebAssembly).
    - Render via `<canvas>`/WebGL.
  - Packaging **NW.js** adds a heavier runtime footprint than a small C++ exe.

### Sketch of the stack

- **Runtime:** **NW.js** (Chromium + **Node.js** in one exe).
- **UI:** HTML/CSS/JS.
- **DDS:** WebGL + a DDS loader (e.g., a **Three.js** DDSLoader or a standalone decoder).
- **JSON:** Native `fs.readFileSync` + `JSON.parse` / `JSON.stringify`.
- **Packaging:** Ship `nw.exe` + `package.json` + your JS/HTML in a folder inside your mod.

This is totally doable, but for what you're describing, it's more moving parts than C++ + Irrlicht.

## Project structure idea (C++ + Irrlicht)

Something like:

```
BATTLETECH/  
  Mods/  
    YourMod/  
      mod.json  
      CustomWeapons/  
      CustomMechs/  
      Editor/  
        bt_mech_editor.exe  
        Irrlicht.dll  
        config.json  
        src/    (for GitHub; not needed for end user)
```

`config.json` might hold:

```
{  
  "gameDataPath": "C:/Program Files (x86)/Steam/steamapps/common/BATTLETECH/BattleTech_Data/StreamingAssets/data",  
  "modPath": "../.."  
}
```

---

## How deep do you want to go?

If you'd like, next step we can:

- Sketch a **minimal C++ + Irrlicht main.cpp** that:
  - Opens a window.
  - Loads one DDS mech image from the game.
  - Displays it with some placeholder text.
- Or outline the **JSON schema mapping** for one mech (mechdef + chassisdef) into C++ structs so you have a clean starting point.

Which path feels more "lightweight and fun" to you right now—C++/Irrlicht or JS/**NW.js**?